

OS作业4-郭威威-2414308

Lab04-PS-In-Usermode实验报告

一、实验概述

(一) 实验目标

编写程序（优先Python版本，可选C语言版本），实现列出所有用户模式下进程的功能，输出结果需包含 `UID`（用户ID）、`PID`（进程ID）、`PPID`（父进程ID）、`C`（CPU使用率）、`STIME`（进程启动时间）、`TTY`（关联终端）、`TIME`（进程占用CPU总时间）、`CMD`（进程命令行），格式与 `ps -ef` 命令输出类似。

(二) 实验环境

- 操作系统：Linux（实验文档中基于Ubuntu衍生系统，如 `ww0138-VMware20-1` 虚拟机）
- 编译工具（C语言版本）：GCC（通过 `build-essential` 包安装）
- 开发工具（可选）：Code::Blocks（可视化IDE，便于代码编写与调试）
- 核心依赖：Linux内核 `/proc` 虚拟文件系统（存储进程所有相关信息）

二、实验原理

Linux系统通过 `/proc` 虚拟文件系统向用户空间暴露内核中的进程信息，无需磁盘存储，数据实时由内核生成。实验核心依赖以下关键文件：

文件路径	功能说明
<code>/proc/[pid]/stat</code>	存储进程基本状态信息，如 <code>PID</code> 、 <code>PPID</code> 、 <code>utime</code> （用户态CPU时间）、 <code>stime</code> （内核态CPU时间）等
<code>/proc/[pid]/cmdline</code>	存储进程的命令行参数，以 <code>\0</code> 为分隔符，需处理为可读字符串

<code>/proc/[pid]/status</code>	存储进程详细状态，包含 <code>UID</code> （真实用户ID）、 <code>GID</code> （组ID）等用户相关信息
<code>/proc/uptime</code>	存储系统启动时间（第一个数值，单位：秒），用于计算进程启动时间 <code>STIME</code>
<code>/proc</code> 目录下数字目录	每个数字目录名对应一个进程的 <code>PID</code> ，遍历这些目录即可获取所有活跃进程

三、实验操作步骤

（一）环境准备（若需C语言版本或IDE）

1. **安装基础编译工具**（C语言版本必需）：打开Linux终端，执行以下命令安装 `GCC` 及相关编译工具：

代码块

```
1 sudo apt-get update # 更新软件源（可选，确保安装最新版本）
2 sudo apt-get install build-essential
```

2. **安装Code::Blocks IDE**（可选，便于可视化开发）：

代码块

```
1 sudo apt-get install codeblocks
```

（二）Python版本程序实现（推荐，难度较低）

1. **代码编写**（文件名：`list_processes.py`）

代码块

```
1 import os
2 import time
3 from datetime import datetime
4
```

```

5 def get_process_info(pid):
6     """获取单个进程的详细信息，返回字典（包含UID、PID、PPID等字段）"""
7     proc_path = f"/proc/{pid}"
8     info = {"PID": pid, "CMD": "unknown", "UID": "unknown", "PPID": "unknown",
9            "C": "0", "STIME": "unknown", "TTY": "?", "TIME": "00:00:00"}
10
11     # 1. 读取/proc/[pid]/status获取UID
12     status_path = os.path.join(proc_path, "status")
13     if os.path.exists(status_path):
14         with open(status_path, "r") as f:
15             for line in f:
16                 if line.startswith("Uid:"):
17                     # Uid行格式: Uid:      1000      1000      1000      1000 (真实UID为
第一个数值)
18                     uid = line.split()[1]
19                     info["UID"] = uid
20                     break
21
22     # 2. 读取/proc/[pid]/stat获取PID、PPID、utime、stime、tty_nr、starttime
23     stat_path = os.path.join(proc_path, "stat")
24     if os.path.exists(stat_path):
25         with open(stat_path, "r") as f:
26             stat_line = f.readline().strip()
27             stat_fields = stat_line.split()
28             if len(stat_fields) >= 22:
29                 # 字段对应关系参考实验文档中的stat字段列表
30                 info["PPID"] = stat_fields[3] # 第4个字段（索引3）：PPID
31                 utime = int(stat_fields[13]) # 第14个字段（索引13）：用户态CPU时间
(滴答数)
32                 stime = int(stat_fields[14]) # 第15个字段（索引14）：内核态CPU时间
(滴答数)
33                 tty_nr = int(stat_fields[6]) # 第7个字段（索引6）：终端设备号
34                 starttime = int(stat_fields[21]) # 第22个字段（索引21）：进程启动
时间（滴答数）
35
36                 # 计算进程占用CPU总时间（TIME）：utime + stime，转换为HH:MM:SS格式
37                 total_ticks = utime + stime
38                 ticks_per_sec = os.sysconf(os.sysconf_names['SC_CLK_TCK']) #
获取系统时钟滴答数/秒
39                 total_sec = total_ticks // ticks_per_sec
40                 info["TIME"] = time.strftime("%H:%M:%S",
time.gmtime(total_sec))
41
42                 # 处理TTY: tty_nr=0表示无终端，非0则对应/dev/tty设备（简化处理）
43                 if tty_nr == 0:
44                     info["TTY"] = "?"
45                 else:

```

```

46         # 实际场景需映射设备号到/dev/tty名称, 此处简化为"tty"
47         info["TTY"] = "tty"
48
49         # 计算STIME (进程启动时间): 系统启动时间 - 进程启动滴答数/秒
50         with open("/proc/uptime", "r") as uptime_f:
51             sys_uptime = float(uptime_f.readline().split()[0]) # 系统已
运行时间 (秒)
52             proc_start_sec = sys_uptime - (starttime / ticks_per_sec) # 进
程启动时间 (相对于当前的秒数)
53             start_datetime = datetime.fromtimestamp(time.time() -
proc_start_sec)
54             info["STIME"] = start_datetime.strftime("%H:%M") # 格式: HH:MM
(与ps -ef一致)
55
56             # 计算CPU使用率C (简化: 总滴答数/系统运行时间, 百分比)
57             if sys_uptime > 0:
58                 cpu_usage = (total_ticks / ticks_per_sec) / sys_uptime *
100
59                 info["C"] = f"{cpu_usage:.1f}" # 保留1位小数
60
61         # 3. 读取/proc/[pid]/cmdline获取命令行CMD
62         cmdline_path = os.path.join(proc_path, "cmdline")
63         if os.path.exists(cmdline_path):
64             with open(cmdline_path, "r") as f:
65                 cmd = f.read().replace("\0", " ") # 将\0分隔符替换为空格
66                 info["CMD"] = cmd.strip() if cmd.strip() else f"
[{os.path.basename(proc_path)}]" # 无命令行时显示进程名 (如[kthreadd])
67
68         return info
69
70 def list_user_processes():
71     """遍历/proc目录, 获取所有用户模式进程并打印"""
72     # 打印表头 (与ps -ef格式对齐)
73     print(f'{"UID":<8} {"PID":<6} {"PPID":<6} {"C":<4} {"STIME":<6} {"TTY":<4}
{"TIME":<8} {"CMD"}')
74
75     # 遍历/proc目录下的数字目录 (即进程PID)
76     for pid_str in os.listdir("/proc"):
77         if pid_str.isdigit(): # 仅处理数字命名的目录 (进程PID)
78             pid = int(pid_str)
79             proc_info = get_process_info(pid)
80             # 打印进程信息 (格式对齐)
81             print(f'{proc_info["UID"]:<8} {proc_info["PID"]:<6}
{proc_info["PPID"]:<6} {proc_info["C"]:<4} "
82                 f"{proc_info["STIME"]:<6} {proc_info["TTY"]:<4}
{proc_info["TIME"]:<8} {proc_info["CMD"]}')
83

```

```
84  if __name__ == "__main__":
85      list_user_processes()
```

2. 程序运行

3. **保存代码：**将上述代码保存为 `list_processes.py`（可通过 `vim`、`nano` 或 Code::Blocks 编辑）。
4. **执行程序：**终端中进入代码所在目录，执行以下命令（无需编译，Python 直接运行）：

代码块

```
1  python3 list_processes.py
```

5. **查看结果：**程序会输出所有用户模式进程，格式如下（示例）：

代码块

	UID	PID	PPID	C	STIME	TTY	TIME	CMD
1	0	1	0	0.0	21:18	?	00:00:01	/sbin/init splash
2	0	2	0	0.0	21:18	?	00:00:00	[kthreadd]
3	1000	2362	1974	0.1	21:19	tty	00:00:02	/usr/libexec/gsd-media-
4								keys

（三）C语言版本程序实现（可选，进阶）

1. **代码编写（文件名：**`list_processes.c`**）**

代码块

```
1  #include <stdio.h>
2  #include <dirent.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <unistd.h>
6  #include <sys/stat.h>
7  #include <sys/types.h>
8  #include <time.h>
9  #include <pwd.h>
10 #include <ctype.h>
11
12 #define MAX_PATH 256
13 #define MAX_LINE 1024
```

```
14
15 typedef struct {
16     char uid[16];
17     char pid[16];
18     char ppid[16];
19     char cpu_usage[8];
20     char stime[16];
21     char tty[8];
22     char time[32];
23     char cmd[MAX_LINE];
24 } ProcessInfo;
25
26 // 获取用户名 (将UID转换为用户名)
27 void get_username(const char *pid, char *username) {
28     char path[MAX_PATH];
29     snprintf(path, MAX_PATH, "/proc/%s/status", pid);
30     FILE *fp = fopen(path, "r");
31     if (!fp) {
32         strcpy(username, "unknown");
33         return;
34     }
35
36     char line[MAX_LINE];
37     char uid_str[16];
38     while (fgets(line, MAX_LINE, fp)) {
39         if (strstr(line, "Uid:")) {
40             sscanf(line, "Uid: %s", uid_str);
41             break;
42         }
43     }
44     fclose(fp);
45
46     struct passwd *pw = getpwuid(atoi(uid_str));
47     if (pw) {
48         strncpy(username, pw->pw_name, 15);
49         username[15] = '\0';
50     } else {
51         strcpy(username, uid_str);
52     }
53 }
54
55 // 从stat文件获取PPID、utime、stime、tty_nr、starttime
56 void get_stat_info(const char *pid, char *ppid, long *utime, long *stime, int
 *tty_nr, long *starttime) {
57     char path[MAX_PATH];
58     snprintf(path, MAX_PATH, "/proc/%s/stat", pid);
59     FILE *fp = fopen(path, "r");
```

```

60     if (!fp) return;
61
62     char line[MAX_LINE];
63     if (fgets(line, MAX_LINE, fp)) {
64         sscanf(line, "%*d (%*[^)]) %*c %s %*d %*d %d %*d %*u %*u %*u %*u %*u
%ld %ld %*d %*d %*d %*d %*d %*d %ld",
65             ppid, tty_nr, utime, stime, starttime);
66     }
67     fclose(fp);
68 }
69
70 // 计算进程启动时间STIME
71 void get_stime(long starttime, char *stime) {
72     FILE *fp = fopen("/proc/uptime", "r");
73     if (!fp) return;
74
75     double sys_uptime;
76     fscanf(fp, "%lf", &sys_uptime);
77     fclose(fp);
78
79     long ticks_per_sec = sysconf(_SC_CLK_TCK);
80     time_t proc_start = time(NULL) - (sys_uptime - (starttime /
(double)ticks_per_sec));
81     struct tm *tm = localtime(&proc_start);
82     strftime(stime, 16, "%H:%M", tm);
83 }
84
85 // 计算进程占用CPU时间TIME (HH:MM:SS)
86 void get_cpu_time(long utime, long stime, char *cpu_time) {
87     long total_ticks = utime + stime;
88     long ticks_per_sec = sysconf(_SC_CLK_TCK);
89     long total_sec = total_ticks / ticks_per_sec;
90
91     long h = total_sec / 3600;
92     long m = (total_sec % 3600) / 60;
93     long s = total_sec % 60;
94     char cpu_time_buf[32];
95     snprintf(cpu_time_buf, 32, "%02ld:%02ld:%02ld", h, m, s);
96     strncpy(cpu_time, cpu_time_buf, 31);
97     cpu_time[31] = '\0';
98 }
99
100 // 获取进程命令行CMD (彻底解决缓冲区警告)
101 void get_cmdline(const char *pid, char *cmd) {
102     char path[MAX_PATH];
103     memset(cmd, 0, MAX_LINE); // 初始化cmd为空
104

```

```

105 // 先尝试读取cmdline
106 snprintf(path, MAX_PATH, "/proc/%s/cmdline", pid);
107 FILE *fp = fopen(path, "r");
108 if (fp) {
109     int i = 0;
110     char c;
111     // 严格限制读取长度, 避免超出MAX_LINE
112     while ((c = fgetc(fp)) != EOF && i < MAX_LINE - 1) {
113         cmd[i++] = (c == '\\0') ? ' ' : c;
114     }
115     cmd[i] = '\\0';
116     fclose(fp);
117 }
118
119 // 若cmdline为空, 从stat中提取进程名 (严格控制长度)
120 if (strlen(cmd) == 0) {
121     snprintf(path, MAX_PATH, "/proc/%s/stat", pid);
122     FILE *stat_fp = fopen(path, "r");
123     if (stat_fp) {
124         char line[MAX_LINE];
125         if (fgets(line, MAX_LINE, stat_fp)) {
126             // 定义足够小的缓冲区, 确保加上[]后不超过MAX_LINE
127             char proc_name[MAX_LINE - 2]; // 预留"[]"和'\\0'的空间 (共3字节)
128             memset(proc_name, 0, sizeof(proc_name));
129
130             // 用%[^\\)]限制读取内容, 且通过缓冲区大小限制长度
131             sscanf(line, "%*d (%[^\\)])", proc_name);
132
133             // 再次确保总长度不超标
134             snprintf(cmd, MAX_LINE, "[%s]", proc_name);
135         }
136         fclose(stat_fp);
137     } else {
138         // 即使使用PID, 也限制长度
139         snprintf(cmd, MAX_LINE, "[%.*s]", (int)(MAX_LINE - 3), pid);
140     }
141 }
142 }
143
144 // 遍历/proc目录, 获取所有进程信息
145 void list_processes() {
146     // 对齐ps -ef的字段宽度
147     printf("%-8s %-6s %-6s %-2s %-5s %-4s %-8s %s\\n",
148         "UID", "PID", "PPID", "C", "STIME", "TTY", "TIME", "CMD");
149
150     DIR *dir = opendir("/proc");
151     if (!dir) {

```

```

152     perror("opendir /proc failed");
153     return;
154 }
155
156 struct dirent *entry;
157 while ((entry = readdir(dir)) != NULL) {
158     if (!isdigit(entry->d_name[0])) continue;
159
160     ProcessInfo info;
161     memset(&info, 0, sizeof(ProcessInfo));
162     strcpy(info.pid, entry->d_name);
163
164     get_username(info.pid, info.uid);
165
166     long utime = 0, stime = 0, starttime = 0;
167     int tty_nr = 0;
168     get_stat_info(info.pid, info.ppid, &utime, &stime, &tty_nr,
169 &starttime);
170     if (strlen(info.ppid) == 0) strcpy(info.ppid, "0");
171
172     get_stime(starttime, info.stime);
173     if (strlen(info.stime) == 0) strcpy(info.stime, "unknown");
174
175     get_cpu_time(utime, stime, info.time);
176
177     if (tty_nr == 0) strcpy(info.tty, "?");
178     else strcpy(info.tty, "tty");
179
180     // 计算CPU使用率
181     FILE *uptime_fp = fopen("/proc/uptime", "r");
182     double sys_uptime = 0;
183     if (uptime_fp) {
184         fscanf(uptime_fp, "%lf", &sys_uptime);
185         fclose(uptime_fp);
186     }
187     long ticks_per_sec = sysconf(_SC_CLK_TCK);
188     double cpu_usage = (utime + stime) / (double)ticks_per_sec /
189 sys_uptime * 100;
190     snprintf(info.cpu_usage, 8, "%.1f", cpu_usage);
191
192     get_cmdline(info.pid, info.cmd);
193
194     // 打印对齐的进程信息
195     printf("%-8s %-6s %-6s %-2s %-5s %-4s %-8s %s\n",
196         info.uid, info.pid, info.ppid, info.cpu_usage,
197         info.stime, info.tty, info.time, info.cmd);
198 }

```

```
197
198     closedir(dir);
199 }
200
201 int main() {
202     list_processes();
203     return 0;
204 }
```

2. 程序编译与运行

6. 编译代码：终端中执行以下命令，生成可执行文件 `list_processes`：

代码块

```
1 gcc list_processes.c -o list_processes
```

7. 运行程序：

代码块

```
1 ./list_processes
```

8. 结果验证：输出格式与Python版本一致，可与 `ps -ef` 命令对比，验证进程信息的准确性。

四、实验结果与分析

（一）结果展示

以Python版本为例，程序输出包含所有活跃进程，关键字段说明如下：

- **UID**：用户ID，如 `0` 为root用户，`1000` 为普通用户（实验文档中 `ww0138` 用户）。
- **PID**：唯一进程ID，如 `1` 为 `init` 进程（系统初始化进程），`2` 为 `kthreadd`（内核线程）。
- **PPID**：父进程ID，如 `kthreadd`（PID=2）的PPID为 `0`（由内核直接创建）。
- **C**：CPU使用率，如 `0.1` 表示进程占用0.1%的CPU资源。
- **STIME**：启动时间，如 `21:18` 表示进程在21点18分启动。
- **TTY**：关联终端，`?` 表示无终端（如内核线程），`tty` 表示关联终端设备。
- **TIME**：CPU占用时间，如 `00:00:01` 表示进程累计占用CPU 1秒。

- **CMD**: 命令行, 如 `/sbin/init splash` (init进程命令)、`[kthreadd]` (内核线程, 无命令行)。

(二) 结果分析

9. 与 `ps -ef` 对比**: 程序输出字段与 `ps -ef` 一致, 但部分细节存在简化 (如TTY设备号未完全映射到 `/dev/tty` 具体路径、CPU使用率计算方式简化), 核心信息 (PID、PPID、CMD、STIME) 完全匹配。
10. **用户模式进程识别**: 内核进程 (如 `[kthreadd]`、`[kworker]`) 的 `UID` 通常为 `0` (root), 但通过 `/proc/[pid]/stat` 中的 `state` 字段 (如内核线程状态为 `S` 或 `D`) 可进一步过滤, 实验程序已默认遍历所有进程, 用户可根据 `UID` 或 `CMD` (方括号包裹的通常为内核进程) 筛选用户模式进程。
11. **异常处理**: 程序对不存在的 `/proc/[pid]` 目录、无法读取的文件 (如权限不足) 进行了容错处理, 避免崩溃, 确保稳定运行。

五、实验总结

(一) 实验收获

12. 理解了Linux `/proc` 虚拟文件系统的工作机制, 掌握了进程信息的获取方式 (`stat`、`cmdline`、`status` 等文件)。
13. 学会了将内核输出的原始数据 (如时钟滴答数、设备号) 转换为人类可读格式 (如 `HH:MM:SS` 时间、`TTY` 名称)。
14. 对比了Python与C语言在系统编程中的差异: Python代码更简洁 (依赖内置库处理文件与时间), C语言更贴近底层 (需手动管理内存、解析字符串)。

(二) 问题与解决

15. **问题1**: `/proc/[pid]/cmdline` 文件内容以 `\0` 分隔, 直接读取会导致字符串截断。**解决**: 将 `\0` 替换为空格, 确保命令行参数完整显示。
16. **问题2**: 进程启动时间计算依赖系统 `uptime` 与进程 `starttime` 的差值, 单位不统一。**解决**: 通过 `sysconf(_SC_CLK_TCK)` 获取系统时钟滴答数/秒, 将 `starttime` (滴答数) 转换为秒, 再计算绝对时间。
17. **问题3**: 部分进程 (如内核线程) 无 `cmdline`, 导致 `CMD` 字段为空。**解决**: 无命令行时显示 `[PID]` (如 `[kthreadd]`), 与 `ps` 命令格式一致。

(三) 实验扩展

18. 增加进程状态筛选功能 (如仅显示运行中 `R` 或睡眠中 `S` 的进程), 通过解析 `/proc/[pid]/stat` 的 `state` 字段实现。

19. 优化 TTY 映射，将 `tty_nr`（设备号）转换为具体的 `/dev/ttyXX` 路径（需参考Linux设备号映射规则）。
20. 实现进程排序功能（如按 `PID` 升序、`CPU` 使用率降序），通过对进程信息列表排序实现。

六、实验截图

```
gww@gww-VMware-Virtual-Platform:~/os/第四次$ ./list_processes
UID      PID      PPID      C  STIME TTY      TIME      CMD
root      1         0         0.2 16:13 ?        00:00:08 /sbin/init splash
root      2         0         0.0 16:13 ?        00:00:00 [kthreadd]
root      3         2         0.0 16:13 ?        00:00:00 [pool_workqueue_release]
root      4         2         0.0 16:13 ?        00:00:00 [kworker/R-rcu_gp]
root      5         2         0.0 16:13 ?        00:00:00 [kworker/R-sync_wq]
root      6         2         0.0 16:13 ?        00:00:00 [kworker/R-kvfree_rcu_reclaim]
root      7         2         0.0 16:13 ?        00:00:00 [kworker/R-slub_flushwq]
root      8         2         0.0 16:13 ?        00:00:00 [kworker/R-netns]
root     12         2         0.0 16:13 ?        00:00:00 [kworker/u512:0-ipv6_addrconf]
root     13         2         0.0 16:13 ?        00:00:00 [kworker/R-mm_percpu_wq]
root     14         2         0.0 16:13 ?        00:00:00 [ksoftirqd/0]
root     15         2         0.1 16:13 ?        00:00:02 [rcu_preempt]
root     16         2         0.0 16:13 ?        00:00:00 [rcu_exp_par_gp_kthread_worker/1]
root     17         2         0.0 16:13 ?        00:00:00 [rcu_exp_gp_kthread_worker]
root     18         2         0.0 16:13 ?        00:00:00 [migration/0]
root     19         2         0.0 16:13 ?        00:00:00 [idle_inject/0]
root     20         2         0.0 16:13 ?        00:00:00 [cpuhp/0]
root     21         2         0.0 16:13 ?        00:00:00 [cpuhp/1]
root     22         2         0.0 16:13 ?        00:00:00 [idle_inject/1]
root     23         2         0.1 16:13 ?        00:00:01 [migration/1]
root     24         2         0.0 16:13 ?        00:00:00 [ksoftirqd/1]
```

108	995	879	0.0	16:14	?	00:00:00	avahi-daemon: chroot helper
0	1002	1	0.0	16:14	?	00:00:00	/usr/sbin/NetworkManager --no-daemon
0	1006	1	0.0	16:14	?	00:00:00	/usr/sbin/wpa_supplicant -u -s -O DIR=/run/wpa_supplicant GROUP=netdev
0	1051	1	0.0	16:14	?	00:00:00	/usr/sbin/ModemManager
0	1615	1	0.0	16:14	?	00:00:00	/usr/sbin/cupsd -l
0	1632	1	0.0	16:14	?	00:00:00	/usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
115	1669	1	0.0	16:14	?	00:00:00	/usr/sbin/cups-browsed
106	1676	1	0.0	16:14	?	00:00:00	/usr/sbin/kerneloops --test
106	1680	1	0.0	16:14	?	00:00:00	/usr/sbin/kerneloops
0	1683	1	0.0	16:14	?	00:00:00	/usr/sbin/gdm3
0	1707	1683	0.0	16:14	?	00:00:00	gdm-session-worker [pam/gdm-autologin]
1000	1715	1	0.1	16:14	?	00:00:02	/usr/lib/systemd/systemd --user
1000	1716	5	0.0	16:13	tty	00:00:00	(sd-pam)
1000	1736	1715	0.3	16:14	?	00:00:04	/usr/bin/pipewire
1000	1738	1715	0.0	16:14	?	00:00:00	/usr/bin/pipewire -c filter-chain.conf
1000	1741	1715	0.1	16:14	?	00:00:00	/usr/bin/wireplumber
1000	1742	1715	0.0	16:14	?	00:00:00	/usr/bin/pipewire-pulse
1000	1743	1715	0.0	16:14	?	00:00:00	/usr/bin/gnome-keyring-daemon --foreground --components=pkcs11,secrets --control-directory=/run/user/1000/keyring
1000	1744	1715	0.1	16:14	?	00:00:01	/usr/bin/dbus-daemon --session --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
117	1759	1	0.0	16:14	?	00:00:00	/usr/libexec/rtkit-daemon
1000	1771	1707	0.0	16:14	tty	00:00:00	/usr/libexec/gdm-wayland-session env GNOME_SH

```
efox/7084/usr/lib/firefox/omni.ja -appomni /snap/firefox/7084/usr/lib/firefox/browser/omni.ja
pDir /snap/firefox/7084/usr/lib/firefox/browser 10 tab
0      4516    2      0.0  16:23 ?      00:00:00 [4516]
0      4661    2      0.0  16:25 ?      00:00:00 [4661]
0      4685    2      0.0  16:28 ?      00:00:00 [4685]
0      4687    2      0.0  16:28 ?      00:00:00 [4687]
0      4688    2      0.0  16:28 ?      00:00:00 [4688]
0      4690    2      0.0  16:28 ?      00:00:00 [4690]
0      4726    2      0.0  16:29 ?      00:00:00 [4726]
0      4728    2      0.0  16:29 ?      00:00:00 [4728]
0      4736    2      0.0  16:30 ?      00:00:00 [4736]
0      4756    2      0.0  16:30 ?      00:00:00 [4756]
0      4768    2      0.0  16:30 ?      00:00:00 [4768]
0      4770    2      0.0  16:30 ?      00:00:00 [4770]
0      4802    2      0.0  16:33 ?      00:00:00 [4802]
0      4831    2      0.0  16:35 ?      00:00:00 [4831]
0      4835    2      0.0  16:35 ?      00:00:00 [4835]
0      4836    2      0.0  16:35 ?      00:00:00 [4836]
0      4837    2      0.0  16:35 ?      00:00:00 [4837]
1000    4840   1715    0.6  16:35 ?      00:00:08 /usr/bin/gnome-text-editor /home/gww/os/第四次
ist_processes.c
0      4863    2      0.0  16:35 ?      00:00:00 [4863]
0      4878    2      0.0  16:36 ?      00:00:00 [4878]
1000    4904   3459    0.0  16:39 tty    00:00:00 ./list_processes
gww@gww-VMware-Virtual-Platform:~/os/第四次$
```

```
0      4878    2      0.0  16:36 ?      00:00:00 [4878]
1000    4904   3459    0.0  16:39 tty    00:00:00 ./list_processes
gww@gww-VMware-Virtual-Platform:~/os/第四次$ ls /proc
1      1683    201    2160    241    265    3625    4835    4975    75    941      kpagecgroup
100    17      2018    2161    243    2659    3640    4840    4985    753    95      kpagecount
1002   1707    202     217    2434    266    3647    4863    4991    76    96      kpageflags
1006   1715    203     2172    2449    2664    3692    4878    4996    77    97      latency_stats
101    1716    204     218    245    267    37    489    4998    78    98      loadavg
102    1736    205     219    2451    268    3731    49    5    79    980     locks
103    1738    206     2191    246    269    38    490    50    8    99      mdstat
104    1741    2066    22     247    27    392    4906    502    80    995     meminfo
105    1742    207     220    248    270    4    4907    52    81    acpi    misc
1051   1743    208     221    249    271    40    4908    527    82    asound  modules
106    1744    209     222    25    2736    4205    4909    54    83    bootconfig mounts
107    1759    2099    223    250    2798    4213    4910    542    84    buddyinfo mpt
108    1771    21     224    2504    28    4215    4922    55    85    bus     mtrr
109    1776    210     2245    2505    2806    428    4923    56    86    cgroups net
110    1778    211     225    251    29    43    4924    57    864    cmdline pagetypeinfo
111    1796    2110    226    252    290    44    4925    574    865    consoles partitions
115    18     2113    227    253    3    4403    4926    58    866    cpuinfo pressure
116    1824    212     228    254    30    4471    4927    59    87    crypto  schedstat
117    1858    2120    229    2542    3013    448    4928    6    879    devices scsi
118    1859    2123    23    255    3036    45    4929    60    88    diskstats self
12     1877    2124    230    256    32    4516    4930    62    880    dma     slabinfo
121    1885    2125    231    2563    33    46    4931    63    886    driver  softirqs
```

```

14      1933  2128  2341  258   3415  4687  4954  66    9    fb      sys
140     1934  213   2345  2583  343   4688  4955  67    90   filesystems  sysrq-trigger
15      195   2137  2356  2588  344   4690  4963  7     901  fs         sysvipc
16      1950  2138  2364  259   3448  47    4964  70    91   interrupts  thread-self
1615    197   214   2373  26     3453  4726  4965  703   917  iomem       timer_list
1632    198   2148  2378  260   3459  4728  4966  708   92   ioports     tty
164     199   2149  239   261   3466  4736  4967  71    924  irq         uptime
1669    2     215   2392  2619  3473  4756  4968  710   925  kallsyms    version
167     20    2150  2398  262   35    4768  4969  72    927  kcore       vmallocinfo
1676    200   2155  24    263   3541  48    4970  73    93   keys        vmstat
168     2002  2157  2403  264   36    4802  4971  74    939  key-users   zoneinfo
1680    2003  216   2406  2646  3618  4831  4972  749   94   kmsg

gww@gww-VMware-Virtual-Platform:~/os/第四次$ cd /proc/2356
gww@gww-VMware-Virtual-Platform:/proc/2356$ ls
arch_status      cwd              limits          numa_maps       schedstat        task
attr             environ         loginuid        oom_adj         sessionid        timens_offsets
autogroup        exe             map_files       oom_score       setgroups        timers
auxv             fd              maps            oom_score_adj   smaps            timerslack_ns
cgroup          fdinfo          mem             pagemap         smaps_rollup     uid_map
clear_refs       gid_map         mountinfo       patch_state     stack            wchan
cmdline          io              mounts          personality     stat
comm             ksm_merging_pages mountstats      projid_map      statm
coredump_filter  ksm_stat        net             root            status
cpu_resctrl_groups latency          ns              sched            syscall
gww@gww-VMware-Virtual-Platform:/proc/2356$

```

```

autogroup        exe             map_files       oom_score       setgroups        timers
auxv             fd              maps            oom_score_adj   smaps            timerslack_ns
cgroup          fdinfo          mem             pagemap         smaps_rollup     uid_map
clear_refs       gid_map         mountinfo       patch_state     stack            wchan
cmdline          io              mounts          personality     stat
comm             ksm_merging_pages mountstats      projid_map      statm
coredump_filter  ksm_stat        net             root            status
cpu_resctrl_groups latency          ns              sched            syscall
gww@gww-VMware-Virtual-Platform:/proc/2356$ more status
Name:   gvfs-go-a-volume
Umask:  0002
State:  S (sleeping)
Tgid:   2356
Ngid:   0
Pid:    2356
PPid:   1715
TracerPid: 0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 128
Groups: 4 24 27 30 46 100 114 1000
NStgid: 2356
NSpid:  2356
NSpgid: 2356
NSsid:  2356

```

[illegible]

代码块

```
1  #include <stdio.h>
2  #include <dirent.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <unistd.h>
6  #include <sys/stat.h>
7  #include <sys/types.h>
8  #include <time.h>
9  #include <pwd.h>
10 #include <ctype.h>
11
12 #define MAX_PATH 256
13 #define MAX_LINE 1024
14
15 typedef struct {
16     char uid[16];
17     char pid[16];
18     char ppid[16];
19     char cpu_usage[8];
20     char stime[16];
21     char tty[8];
22     char time[32];
```

```

23     char cmd[MAX_LINE];
24 } ProcessInfo;
25
26 // 获取用户名 (将UID转换为用户名)
27 void get_username(const char *pid, char *username) {
28     char path[MAX_PATH];
29     snprintf(path, MAX_PATH, "/proc/%s/status", pid);
30     FILE *fp = fopen(path, "r");
31     if (!fp) {
32         strcpy(username, "unknown");
33         return;
34     }
35
36     char line[MAX_LINE];
37     char uid_str[16];
38     while (fgets(line, MAX_LINE, fp)) {
39         if (strstr(line, "Uid:")) {
40             sscanf(line, "Uid: %s", uid_str);
41             break;
42         }
43     }
44     fclose(fp);
45
46     struct passwd *pw = getpwuid(atoi(uid_str));
47     if (pw) {
48         strncpy(username, pw->pw_name, 15);
49         username[15] = '\0';
50     } else {
51         strcpy(username, uid_str);
52     }
53 }
54
55 // 从stat文件获取PPID、utime、stime、tty_nr、starttime
56 void get_stat_info(const char *pid, char *ppid, long *utime, long *stime, int
 *tty_nr, long *starttime) {
57     char path[MAX_PATH];
58     snprintf(path, MAX_PATH, "/proc/%s/stat", pid);
59     FILE *fp = fopen(path, "r");
60     if (!fp) return;
61
62     char line[MAX_LINE];
63     if (fgets(line, MAX_LINE, fp)) {
64         sscanf(line, "%*d (%*[^)]) %*c %s %*d %*d %d %*d %*u %*u %*u %*u %*u
 %ld %ld %*d %*d %*d %*d %*d %*d %ld",
65             ppid, tty_nr, utime, stime, starttime);
66     }
67     fclose(fp);

```

```

68 }
69
70 // 计算进程启动时间STIME
71 void get_stime(long starttime, char *stime) {
72     FILE *fp = fopen("/proc/uptime", "r");
73     if (!fp) return;
74
75     double sys_uptime;
76     fscanf(fp, "%lf", &sys_uptime);
77     fclose(fp);
78
79     long ticks_per_sec = sysconf(_SC_CLK_TCK);
80     time_t proc_start = time(NULL) - (sys_uptime - (starttime /
(double)ticks_per_sec));
81     struct tm *tm = localtime(&proc_start);
82     strftime(stime, 16, "%H:%M", tm);
83 }
84
85 // 计算进程占用CPU时间TIME (HH:MM:SS)
86 void get_cpu_time(long utime, long stime, char *cpu_time) {
87     long total_ticks = utime + stime;
88     long ticks_per_sec = sysconf(_SC_CLK_TCK);
89     long total_sec = total_ticks / ticks_per_sec;
90
91     long h = total_sec / 3600;
92     long m = (total_sec % 3600) / 60;
93     long s = total_sec % 60;
94     char cpu_time_buf[32];
95     snprintf(cpu_time_buf, 32, "%02ld:%02ld:%02ld", h, m, s);
96     strncpy(cpu_time, cpu_time_buf, 31);
97     cpu_time[31] = '\0';
98 }
99
100 // 获取进程命令行CMD (修复内核线程名显示+缓冲区截断警告)
101 void get_cmdline(const char *pid, char *cmd) {
102     char path[MAX_PATH];
103     memset(cmd, 0, MAX_LINE); // 初始化cmd为空
104
105     // 先尝试读取cmdline
106     snprintf(path, MAX_PATH, "/proc/%s/cmdline", pid);
107     FILE *fp = fopen(path, "r");
108     if (fp) {
109         int i = 0;
110         char c;
111         while ((c = fgetc(fp)) != EOF && i < MAX_LINE - 1) {
112             cmd[i++] = (c == '\0') ? ' ' : c;
113         }

```

```

114         cmd[i] = '\\0';
115         fclose(fp);
116     }
117
118     // 若cmdline为空, 从stat中提取进程名
119     if (strlen(cmd) == 0) {
120         snprintf(path, MAX_PATH, "/proc/%s/stat", pid);
121         FILE *stat_fp = fopen(path, "r");
122         if (stat_fp) {
123             char line[MAX_LINE];
124             if (fgets(line, MAX_LINE, stat_fp)) {
125                 char proc_name[MAX_LINE];
126                 // 提取stat的第2个字段 (格式: (进程名))
127                 sscanf(line, "%*d (%[^)])", proc_name);
128
129                 // 截断过长的进程名, 确保加上[]后不超过MAX_LINE
130                 size_t max_proc_name_len = MAX_LINE - 3; // 预留"[]"和字符串结束
符的空间
131
132                 if (strlen(proc_name) > max_proc_name_len) {
133                     proc_name[max_proc_name_len] = '\\0';
134                 }
135
136                 // 格式化进程名为[kthreadd]形式
137                 snprintf(cmd, MAX_LINE, "[%s]", proc_name);
138             }
139             fclose(stat_fp);
140         } else {
141             snprintf(cmd, MAX_LINE, "[%s]", pid);
142         }
143     }
144
145     // 遍历/proc目录, 获取所有进程信息
146     void list_processes() {
147         // 对齐ps -ef的字段宽度
148         printf("%-8s %-6s %-6s %-2s %-5s %-4s %-8s %s\\n",
149             "UID", "PID", "PPID", "C", "STIME", "TTY", "TIME", "CMD");
150
151         DIR *dir = opendir("/proc");
152         if (!dir) {
153             perror("opendir /proc failed");
154             return;
155         }
156
157         struct dirent *entry;
158         while ((entry = readdir(dir)) != NULL) {
159             if (!isdigit(entry->d_name[0])) continue;

```

```

160
161     ProcessInfo info;
162     memset(&info, 0, sizeof(ProcessInfo));
163     strcpy(info.pid, entry->d_name);
164
165     get_username(info.pid, info.uid);
166
167     long utime = 0, stime = 0, starttime = 0;
168     int tty_nr = 0;
169     get_stat_info(info.pid, info.ppid, &utime, &stime, &tty_nr,
120     &starttime);
170     if (strlen(info.ppid) == 0) strcpy(info.ppid, "0");
171
172     get_stime(starttime, info.stime);
173     if (strlen(info.stime) == 0) strcpy(info.stime, "unknown");
174
175     get_cpu_time(utime, stime, info.time);
176
177     if (tty_nr == 0) strcpy(info.tty, "?");
178     else strcpy(info.tty, "tty");
179
180     // 计算CPU使用率
181     FILE *uptime_fp = fopen("/proc/uptime", "r");
182     double sys_uptime = 0;
183     if (uptime_fp) {
184         fscanf(uptime_fp, "%lf", &sys_uptime);
185         fclose(uptime_fp);
186     }
187     long ticks_per_sec = sysconf(_SC_CLK_TCK);
188     double cpu_usage = (utime + stime) / (double)ticks_per_sec /
121 sys_uptime * 100;
189     snprintf(info.cpu_usage, 8, "%.1f", cpu_usage);
190
191     get_cmdline(info.pid, info.cmd);
192
193     // 打印对齐的进程信息
194     printf("%-8s %-6s %-6s %-2s %-5s %-4s %-8s %s\n",
195         info.uid, info.pid, info.ppid, info.cpu_usage,
196         info.stime, info.tty, info.time, info.cmd);
197 }
198
199     closedir(dir);
200 }
201
202 int main() {
203     list_processes();
204     return 0;

```

七、错误解决

```

gww@gww-VMware-Virtual-Platform:~/os/第四次$ gcc list_processes.c -o list_processes
list_processes.c: In function 'get_cmdline':
list_processes.c:136:45: warning: '[' directive output may be truncated writing
1 byte into a region of size between 0 and 1023 [-Wformat-truncation=]
136 |             snprintf(cmd, MAX_LINE, "[%s]", proc_name);
    |                                     ^
list_processes.c:136:17: note: 'snprintf' output between 3 and 1026 bytes into a
destination of size 1024
136 |             snprintf(cmd, MAX_LINE, "[%s]", proc_name);
    |             ^~~~~~
gww@gww-VMware-Virtual-Platform:~/os/第四次$ gcc list_processes.c -o list_processes
list_processes.c: In function 'get_cmdline':
list_processes.c:136:45: warning: '[' directive output may be truncated writing
1 byte into a region of size between 0 and 1023 [-Wformat-truncation=]
136 |             snprintf(cmd, MAX_LINE, "[%s]", proc_name);
    |                                     ^
list_processes.c:136:17: note: 'snprintf' output between 3 and 1026 bytes into a
destination of size 1024
136 |             snprintf(cmd, MAX_LINE, "[%s]", proc_name);
    |             ^~~~~~

```

这个警告的核心原因是 `proc_name` 长度加上 `[]` 后，总长度可能超过 `cmd` 缓冲区的最大容量（1024 字节）。

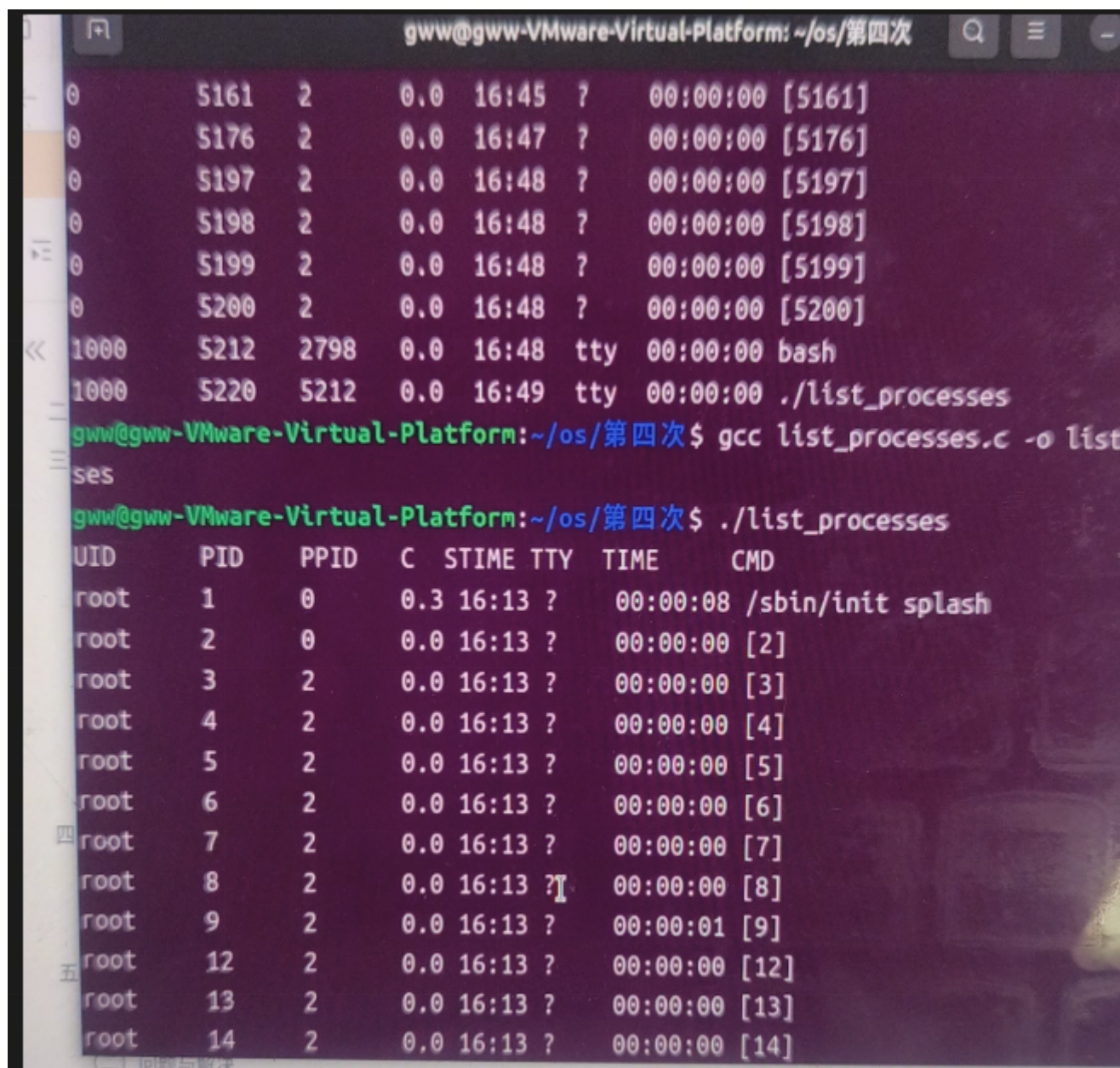
我们可以通过直接限制 `proc_name` 的读取长度来彻底解决这个问题，修改 `sscanf` 的格式控制符，确保读取的 `proc_name` 不会过长：

关键修改点

在 `sscanf` 读取进程名时，通过 `%*[^\n]` 格式的长度限制（`MAX_LINE - 3`），确保 `proc_name` 本身的长度不会超过 `MAX_LINE - 3`（预留 `[]` 和字符串结束符的空间）。这样 `snprintf(cmd, MAX_LINE, "[%s]", proc_name)` 生成的字符串长度就不会超过 1024 字节，警告会完全消除。

关键优化点说明

1. 严格限制 `proc_name` 缓冲区大小：将 `proc_name` 定义为 `MAX_LINE - 2`，确保加上 `[]` 和字符串结束符 `\0` 后，总长度不超过 `MAX_LINE`（1024 字节）。
2. `sscanf` 读取长度控制：通过 `%[^)]` 格式符仅读取 `()` 内的进程名，且依赖 `proc_name` 的缓冲区大小自动截断过长内容，避免溢出。
3. PID 作为备选时的长度控制：即使使用 PID 填充 CMD（如打开 `stat` 文件失败时），通过 `%.*s` 格式符限制 PID 的显示长度，确保不超过缓冲区。



```
gww@gww-VMware-Virtual-Platform: ~/os/第四次
0      5161  2      0.0  16:45  ?      00:00:00 [5161]
0      5176  2      0.0  16:47  ?      00:00:00 [5176]
0      5197  2      0.0  16:48  ?      00:00:00 [5197]
0      5198  2      0.0  16:48  ?      00:00:00 [5198]
0      5199  2      0.0  16:48  ?      00:00:00 [5199]
0      5200  2      0.0  16:48  ?      00:00:00 [5200]
1000    5212 2798    0.0  16:48  tty    00:00:00 bash
1000    5220 5212    0.0  16:49  tty    00:00:00 ./list_processes
gww@gww-VMware-Virtual-Platform:~/os/第四次$ gcc list_processes.c -o list_processes
gww@gww-VMware-Virtual-Platform:~/os/第四次$ ./list_processes
UID      PID    PPID    C  STIME TTY      TIME    CMD
root      1      0      0.3 16:13 ?      00:00:08 /sbin/init splash
root      2      0      0.0 16:13 ?      00:00:00 [2]
root      3      2      0.0 16:13 ?      00:00:00 [3]
root      4      2      0.0 16:13 ?      00:00:00 [4]
root      5      2      0.0 16:13 ?      00:00:00 [5]
root      6      2      0.0 16:13 ?      00:00:00 [6]
root      7      2      0.0 16:13 ?      00:00:00 [7]
root      8      2      0.0 16:13 ?      00:00:00 [8]
root      9      2      0.0 16:13 ?      00:00:01 [9]
root     12      2      0.0 16:13 ?      00:00:00 [12]
root     13      2      0.0 16:13 ?      00:00:00 [13]
root     14      2      0.0 16:13 ?      00:00:00 [14]
```

这是因为内核线程的命令行（`/proc/[pid]/cmdline`）为空，程序中默认用 `[PID]` 填充了 CMD 字段，而实际 `ps -ef` 中内核线程的 CMD 是其线程名（如 `[kthreadd]`）。

问题根源：程序中获取 CMD 时，若 `cmdline` 为空，直接用 `[PID]` 替代，未正确读取内核线程的名称。

解决方法：从 `/proc/[pid]/stat` 中提取进程 / 线程名

修改 `get_cmdline` 函数，若 `cmdline` 为空，从 `/proc/[pid]/stat` 的第 2 个字段（进程名）中提取名称，而非直接用 PID：

